

Latent Continuous Reasoning for Retrieval-Augmented Knowledge Graph Question Answering

Anonymous Author(s)

Abstract

Knowledge graph (KG) question answering (KGQA) requires multi-hop reasoning over retrieved subgraphs to identify answer entities. Recent systems that combine KGs with large language models (LLMs), so-called KG-LLM systems, use generated paths, rationales, or tools for flexibility, but add generation and LLM-call overhead. Graph-native reasoners are efficient, yet they lack persistent global state across recursive propagation. We propose **Latent Continuous Reasoning (LCR)**, which maintains a persistent latent module inside KG inference and reinjects it into instruction modulation, node-state gating, and answer scoring. On ComplexWebQuestions (CWQ), LCR improves Hit@1 from 46.1 to 57.2 and F1 from 43.1 to 55.8 over a carefully matched no-latent training baseline. LCR also remains efficient at ~84 ms per question—roughly 8× faster than the fastest KG-LLM baseline and over 1,700× faster than retrieval-and-traverse LLM methods. Code will be made publicly available upon acceptance.

CCS Concepts

• **Information systems** → *Question answering*; • **Computing methodologies** → *Natural language processing*; **Knowledge representation and reasoning**; *Neural networks*.

Keywords

knowledge graph question answering, multi-hop reasoning, graph neural networks, retrieval-augmented reasoning, latent state maintenance

ACM Reference Format:

Anonymous Author(s). 2026. Latent Continuous Reasoning for Retrieval-Augmented Knowledge Graph Question Answering. In *Proceedings of The 35th ACM International Conference on Information and Knowledge Management (CIKM '26)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Knowledge graph question answering (KGQA) grounds a natural-language question in an explicit graph of entities and relations and returns one or more answer entities. A typical KGQA system first identifies question-relevant entities, retrieves a local subgraph, and then reasons over connected relations to rank candidate answers.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '26, Rome, Italy

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

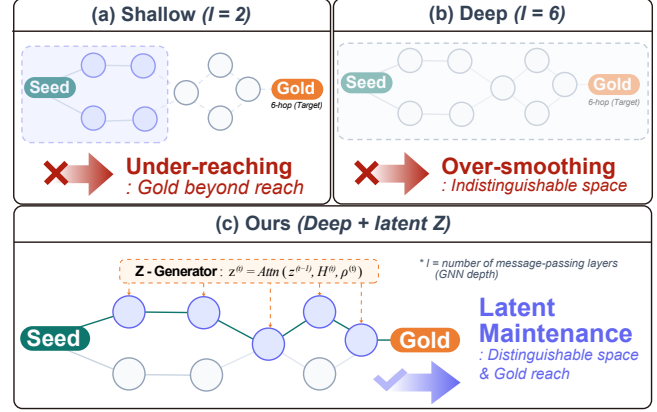


Figure 1: Motivation of latent state maintenance in multi-hop KGQA. Shallow propagation may under-reach distant answers, while deeper propagation improves reachability but can over-smooth candidate representations. LCR keeps the deep reasoning depth and maintains a persistent latent state updated from node states and the answer distribution, preserving global context across propagation steps.

Standard benchmarks such as WebQuestionsSP (WebQSP) and ComplexWebQuestions (CWQ) require models to reason over linked entities, retrieved subgraphs, and compositional constraints [16, 19]. In multi-hop questions, the model must not only reach distant answer nodes but also preserve the intermediate evidence that makes each hop meaningful.

Existing KGQA systems address this challenge through two broad families. Graph-native reasoners propagate question-aware signals directly over KG subgraphs [3, 8, 11–14]. These models are efficient and naturally tied to graph evidence, but their reasoning state is usually carried by local node distributions and recurrent message passing. Recent KG-LLM systems instead externalize intermediate reasoning into generated paths, retrieved contexts, textual rationales, or agentic exploration traces [4, 6, 7, 9, 18]. Such designs improve reasoning flexibility and structural grounding, but they often involve iterative generation, retrieval, or LLM-agent interaction. Recent efficient KGQA studies explicitly identify costly LLM calls, token consumption, and agent-model routing cost as bottlenecks [2, 9].

This contrast motivates a complementary question: can a lightweight graph-native reasoner maintain multi-hop reasoning context internally, without carrying state through repeated generated text or per-query LLM calls? We argue that a key bottleneck is not graph reachability alone, but also maintaining discriminative reasoning state across recursive propagation. Increasing the message-passing depth expands the model’s receptive field and allows information from more distant entities to reach the answer scorer. However,

repeated message passing also aggregates neighboring entity representations at each step, which can make candidate answer representations and scores less distinguishable [1, 5, 10]. As illustrated in Figure 1, the same propagation depth that helps reach a distant answer can reduce the score margin needed to distinguish it from nearby distractors. A persistent latent state offers a way out: it lets the reasoner keep the deep propagation needed to reach distant entities while carrying a global summary that keeps answer scores discriminative. We propose **Latent Continuous Reasoning (LCR)**, which maintains a persistent latent module within the recursive KG inference loop and reinjects it into instruction modulation, node-state gating, and answer scoring. Our contributions are as follows:

- **(1) State Maintenance in Graph-native KGQA:** We focus on recursive state maintenance, an important yet underexplored bottleneck in efficient multi-hop KGQA, and address the challenge of preserving intermediate reasoning evidence across graph propagation steps.
- **(2) Latent Continuous Reasoning Framework:** We introduce LCR, a graph-native latent reasoning framework that maintains a persistent latent module and injects it into instruction modulation, node-state gating, and answer scoring while preserving a fixed ReaRev-style backbone.
- **(3) Empirical Validation:** Experiments on CWQ and WebQSP under matched latent and no-latent training settings demonstrate that a persistent latent module improves answer localization, with complementary gains from each latent injection component.

2 Related Work

Graph-native KGQA. Graph-native methods, often based on graph neural networks (GNNs), reason directly over KG structure: EmbedKGQA scores candidates from learned KG and question representations [11], while GraftNet and PullNet propagate question-aware signals over a question-specific subgraph [13, 14], and later work adds intermediate supervision and adaptive instruction execution [3, 8, 12]. These reasoners are efficient and structurally grounded, but carry reasoning context only in local node states and answer distributions. LCR keeps this graph-native efficiency on the ReaRev backbone [8] and adds a persistent latent state that preserves global context across recursive steps.

KG-LLM reasoning. KG-LLM systems gain flexibility by externalizing reasoning into generated rationales, relation paths, or exploration traces [6, 7, 15, 17, 18]. Closest to us, GNN-RAG retrieves candidate answers and paths with a lightweight GNN before verbalizing them for an LLM [9]. Such methods confirm the value of graph-side evidence but depend on per-query LLM generation. LCR instead strengthens the graph reasoner itself, preserving multi-hop state without routing it through generated text.

Depth and state maintenance. Deeper message passing reaches distant entities but over-smooths node representations, making candidates less distinguishable [1, 5, 10]. Graph transformers add global structure via a static graph-level readout [20], but this summary is computed once, not refreshed during recursive inference. LCR closes this gap: its latent state is updated from the current graph

state and answer distribution at every step and reinjected into the loop, decoupling state maintenance from propagation depth.

3 Preliminaries

Let $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ be a knowledge graph with entities, relations, and triples. Given a question q , retrieval-augmented KGQA first identifies seed entities $\mathcal{S}_q$ and retrieves a question-specific subgraph $\mathcal{G}_q = (\mathcal{V}_q, \mathcal{E}_q)$. The model then ranks candidate entities $v \in \mathcal{V}_q$ by producing an answer distribution $\rho \in \mathbb{R}^{|\mathcal{V}_q|}$.

Graph-native recursive reasoning. Graph-native KGQA answers q by recursively propagating question-aware signals over $\mathcal{G}_q$. From the question, the backbone derives K instruction vectors $\{\mathbf{i}_k\}_{k=1}^K$ that specify what to attend to at each hop. The reasoner then runs for r reasoning steps indexed by $t = 1, \dots, r$, the effective reasoning depth. At step t , it maintains entity states $\mathbf{H}^{(t)} = \{\mathbf{h}_v^{(t)}\}_{v \in \mathcal{V}_q}$ and an answer distribution $\rho^{(t)}$: relation-aware messages conditioned on $\{\mathbf{i}_k\}$ form a candidate update $\hat{\mathbf{h}}_v^{(t)}$, which is fused into $\mathbf{h}_v^{(t)}$ and scored to yield $\rho^{(t)}$.

4 Latent Continuous Reasoning

4.1 Overview

LCR augments a fixed ReaRev-style recursive graph reasoner with a persistent latent state $\mathbf{z}^{(t)} \in \mathbb{R}^{d_z}$, initialized from the projected question as $\mathbf{z}^{(0)} = W_q \mathbf{q}$. Unlike standard graph-native reasoners that carry context only through local node states and answer distributions, LCR maintains this latent state across the entire recursive inference. At each step, the state is injected at three points of the reasoning loop—relation instruction modulation, node-state gating, and answer scoring—and is then updated from the graph, so that global context both shapes and is shaped by every reasoning step.

4.2 Latent Injection and Update

Instruction modulation (Eq. 1). The base instructions $\mathbf{i}_k$ are computed once from the question and stay fixed across hops, which prevents later rounds from re-prioritizing relations as evidence accumulates. LCR therefore modulates each instruction with the previous latent state:

$$\tilde{\mathbf{i}}_k^{(t)} = \mathbf{i}_k + \alpha \tanh\left(W_{\rightarrow i}^{(k)} \mathbf{z}^{(t-1)}\right), \quad (1)$$

$\alpha=0.25$ is a fixed scalar weighting the latent contribution. The modulated instruction $\tilde{\mathbf{i}}_k^{(t)}$ then replaces $\mathbf{i}_k$ when forming the candidate update $\hat{\mathbf{h}}_v^{(t)}$, so later hops refocus relation selection using accumulated reasoning context.

Node-state gating (Eqs. 2–3). Deep propagation can let noisy aggregated messages overwrite evidence that was already well supported, blurring strong candidates. LCR therefore gates each node update with a latent-conditioned gate that decides, per node, how much of the new candidate update to admit:

$$\mathbf{g}_v^{(t)} = \sigma\left(W_g[\mathbf{h}_v^{(t-1)}; \mathbf{z}^{(t-1)}]\right), \quad (2)$$

$$\mathbf{h}_v^{(t)} = \mathbf{g}_v^{(t)} \odot \hat{\mathbf{h}}_v^{(t)} + (1 - \mathbf{g}_v^{(t)}) \odot \mathbf{h}_v^{(t-1)}, \quad (3)$$

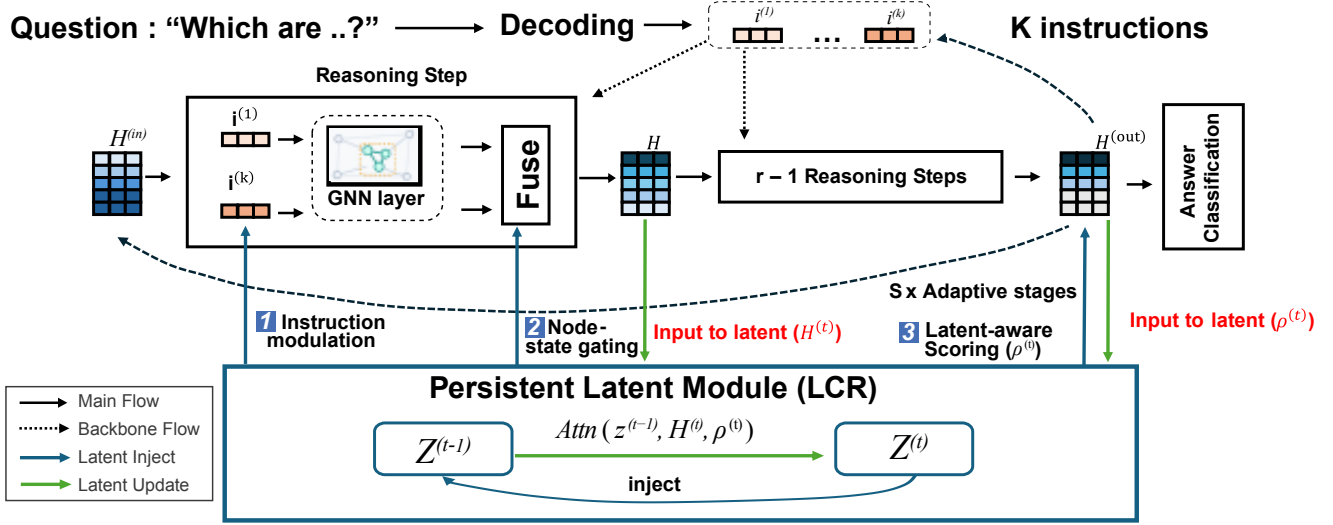


Figure 2: LCR architecture. A ReaRev-style recursive reasoner is augmented with a persistent latent state $z^{(t)}$. At each round, $z^{(t-1)}$ is injected into instruction modulation, node-state gating, and latent-aware scoring, then updated from the current node states and answer distribution.

where $\hat{h}_v^{(t)}$ is the base reasoner’s candidate update from the aggregated messages. The gate thus balances the incoming message against the retained state, conditioned on global context.

Latent-aware scoring (Eq. 4). The base scorer ranks candidates from the local node state alone, which ignores evidence beyond the immediate neighborhood. LCR augments scoring with the latent state so that ranking can account for the global reasoning context:

$$s_v^{(t)} = \mathbf{w}_o^\top \mathbf{h}_v^{(t)} + \psi\left([\mathbf{h}_v^{(t)} \parallel \mathbf{z}^{(t-1)} \parallel \mathbf{h}_v^{(t)} \mathbf{z}^{(t-1)}]\right), \quad (4)$$

where the first term preserves the base local scorer and the second adjusts the ranking with global context. The answer distribution is $\rho^{(t)} = \text{softmax}(\mathbf{s}^{(t)})$.

Latent update (Eq. 5). After scoring, LCR refreshes the latent by attending over the entity states with the answer distribution as a prior,

$$\mathbf{z}^{(t)} = \text{Attn}(\mathbf{z}^{(t-1)}, \mathbf{H}^{(t)}, \rho^{(t)}), \quad (5)$$

where $\mathbf{z}^{(t-1)}$ is the query, the entity states $\mathbf{H}^{(t)}$ are keys/values, and W_Q, W_K, W_V are the corresponding projections (d : key dimension; LN: LayerNorm):

$$\mathbf{c}^{(t)} = \sum_v \text{softmax}_v\left(\frac{(W_Q \mathbf{z}^{(t-1)})^\top W_K \mathbf{h}_v^{(t)}}{\sqrt{d}} + \log \rho^{(t)}(v)\right) W_V \mathbf{h}_v^{(t)}, \quad (6)$$

$$\mathbf{z}^{(t)} = \text{LN}\left((1 - \beta^{(t)}) \mathbf{z}^{(t-1)} + \beta^{(t)} \mathbf{c}^{(t)}\right), \quad \beta^{(t)} = \sigma(\mathbf{w}_\beta^\top [\mathbf{z}^{(t-1)}; \mathbf{c}^{(t)}]) \quad (7)$$

The prior focuses the read on reasoner-important nodes; the gated residual keeps $\mathbf{z}^{(t)}$ persistent across depth.

4.3 Training Objective

LCR reuses the ReaRev-style reasoning architecture but pairs it with our own retrieval setting and training schedule; the matched no-latent baseline shares this exact setup, so any comparison isolates the effect of the persistent latent state. Training follows two phases.

Let $\mathcal{A}_q \subseteq \mathcal{V}_q$ be the gold answers in the retrieved subgraph. Phase I provides broad answer localization by minimizing Kullback–Leibler (KL) divergence

$$\mathcal{L}_{P1} = \text{KL}(u_q \parallel \rho^{(r)}), \quad (8)$$

where u_q is uniform over $\mathcal{A}_q$. Phase II then sharpens candidate ranking with

$$\mathcal{L}_{P2} = \mathcal{L}_{\text{BCE}} + \lambda \mathcal{L}_{\text{rank}}, \quad (9)$$

a smoothed binary cross-entropy (BCE) plus a margin-ranking loss over hard non-answers ($\lambda=0.15$).

5 Experiments

5.1 Setup

Benchmarks. We evaluate on CWQ [16] as the main multi-hop benchmark and WebQSP [19] as a shallower complementary benchmark. We report Hit@1 and F1 over the candidate answer set.

Implementation. Subgraphs are capped at 2,048 nodes / 8,192 edges to fit training on a single 12 GB GPU. This budget is tighter than the retrieval used by the original ReaRev [8] and GNN-RAG [9] pipelines, which lowers the absolute Hit@1 ceiling for *all* models we train. We use $S=2$ stages and $r=6$ reasoning steps—so the latent state is refreshed $S * r=12$ times per question and persists across stages—with $K=3$ instructions, hidden dimension $d=512$ and dropout 0.1.

Retrieval coverage. The gold answer is retained for 80.6% (CWQ) and 95.6% (WebQSP) of questions under our 2,048/8,192 budget, which upper-bounds Hit@1 and accounts for the lower absolute ceiling relative to systems that use larger retrieval.

5.2 Main Results against KGQA Systems

Table 1 positions LCR-Attn against representative KGQA systems; among graph-native methods it achieves strong CWQ performance, reaching 57.2 Hit@1 and 55.8 F1 under the tighter retrieval budget. On WebQSP, LCR improves over its matched no-latent baseline but

Table 1: KGQA performance on WebQSP and CWQ. Bold/underline mark the best per column/family. Hit denotes lenient any-match and Hit@1 strict top-1.

Family	Method	WebQSP			CWQ		
		Hit	Hit@1	F1	Hit	Hit@1	F1
Embedding	EmbedKGQA [11]	–	66.6	–	–	–	–
	TransferNet [12]	–	<u>71.4</u>	–	–	<u>48.6</u>	–
GNN	NSM [3]	–	68.7	62.8	–	47.6	42.4
	SR+NSM(+E2E) [21]	–	69.5	64.1	–	50.2	47.1
	ReaRev [8]	–	<u>76.4</u>	<u>70.9</u>	–	<u>52.9</u>	<u>47.8</u>
LLM-only	ChatGPT	66.8	–	–	39.9	–	–
	ChatGPT+CoT [18]	<u>75.6</u>	–	–	<u>48.9</u>	–	–
KG-LLM	KD-CoT [17]	68.6	–	52.5	55.7	–	–
	UniKGQA [4]	–	77.2	<u>72.2</u>	–	51.2	49.1
	ToG (ChatGPT) [15]	76.2	–	–	58.9	–	–
	GNN-RAG [9]	<u>85.7</u>	<u>80.6</u>	71.3	66.8	<u>61.7</u>	<u>59.4</u>
	RoG [6]	<u>85.7</u>	80.0	70.8	62.6	57.8	56.2
	Ours [†]	LCR-Attn (graph-native)	–	69.0	62.7	–	57.2

[†] Averaged over three seeds; std. ≤ 0.3 . Our rows use a 2,048/8,192 subgraph cap, while cited rows use their original retrieval settings. LCR-Attn denotes LCR with the attention-based latent update in Eq. 5.

Table 2: Effect of latent maintenance. All rows share the same backbone, retrieval budget, and training schedule.

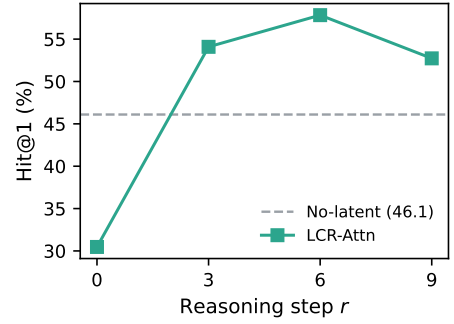
	Hit@1	F1
<i>CWQ</i>		
No latent (baseline)	46.1	43.1
Instruction modulation (Eq. 1)	50.9	49.4
Node-state gating (Eq. 3)	52.5	51.2
Latent-aware scoring (Eq. 4)	52.7	51.1
LCR-Attn (all three sites)	57.2	55.8
<i>WebQSP</i>		
No latent	63.4	53.6
LCR-Attn	69.0	62.7

remains below the strongest published systems, consistent with WebQSP being shallower and leaving less room for persistent state maintenance. KG-LLM systems remain stronger on several metrics, but require one or more LLM passes per query.

5.3 Controlled Ablations

Table 2 reports the controlled paired comparison. All rows share the same backbone, subgraph budget, and training pipeline, so the gap between the no-latent baseline and LCR isolates the effect of latent maintenance. LCR yields the largest gains on CWQ and smaller but consistent gains on WebQSP, matching the intuition that persistent state is more useful for compositional multi-hop questions. The three injection sites are also complementary: each improves over the baseline, while combining all three achieves the best result.

Depth and over-smoothing. Figure 3 sweeps the reasoning depth $r \in \{0, 3, 6, 9\}$ on CWQ. Hit@1 increases up to $r=6$ as propagation reaches more distant evidence, but falls at $r=9$, consistent with over-smoothing making candidate representations less distinguishable [1, 5, 10]. Across $r \geq 3$, LCR remains above the matched

**Figure 3: CWQ Hit@1 across reasoning depth r . LCR peaks at $r=6$ and drops at $r=9$, suggesting over-smoothing.****Table 3: Single-GPU latency on CWQ. LCR latency includes query encoding (66 ms reasoning + 18 ms encoding); entity/relation embeddings are precomputed offline. #Calls is the average per-query LLM calls.**

Method	Params	ms	vs. LCR	#Calls
<i>KG-LLM (official)</i>				
ToG	8 B	146,581	1,745×	20.9
RoG	6.7 B	2,527	30×	2.0
GNN-RAG	6.7 B	689	8×	1.0
<i>Graph-native / hybrid (ours)</i>				
LCR-Attn + RoG-7B	+6.7 B	1,191	14×	1.0
LCR-Attn	23.1 M	84	1×	0

no-latent baseline, indicating that the persistent latent state helps preserve discriminative context during deep propagation.

5.4 Efficiency and Portability

Table 3 quantifies the latency gap on CWQ. With only 23.1M parameters and no per-query LLM call, LCR-Attn answers in 84 ms, including query encoding. This is about 8× faster than GNN-RAG and over 1,700× faster than ToG, which averages 20.9 LLM calls per question.

LCR complements rather than replaces KG-LLM pipelines: operating on the graph side, it can be paired with RoG-7B at the latency in Table 3 without extra per-query LLM calls. A full accuracy evaluation of this hybrid is left to future work.

6 Conclusion

This work introduces LCR, a graph-native KGQA framework that maintains a persistent latent state across recursive inference. Across controlled ablations, LCR improves multi-hop answer localization, with the largest gain on CWQ and complementary contributions from all three injection sites. These results suggest that recursive latent maintenance is a practical lever for graph-side multi-hop reasoning under tight latency budgets. Future work will pair LCR with stronger retrieval and LLM-reader pipelines.

GenAI Usage Disclosure

In accordance with related policies, we disclose the use of a generative AI tool (Claude Code) during the preparation of this manuscript. For language editing, it was employed only to check and correct grammatical errors and typos, improve sentence clarity, and ensure a consistent academic writing style. Separately, it was used for debugging existing code, understanding third-party code, and LaTeX formatting; it was not used to generate scientific claims, experimental results, or manuscript text. All AI-processed text was carefully reviewed and approved by the authors, with the research content, methodology, results, and conclusions remaining entirely human-generated. No generative AI was used in the research design, data analysis, or technical interpretation of results. The final manuscript represents the original intellectual contributions of the authors alone.

References

- [1] Chen Cai and Yusu Wang. 2020. A note on over-smoothing for graph neural networks. *arXiv preprint arXiv:2006.13318* (2020).
- [2] Junnan Dong, Qinggang Zhang, Chuang Zhou, Hao Chen, Daochen Zha, and Xiao Huang. 2024. Cost-efficient knowledge-based question answering with large language models. *Advances in Neural Information Processing Systems* 37 (2024), 115261–115281.
- [3] Gaole He, Yunshi Lan, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. 2021. Improving multi-hop knowledge base question answering by learning intermediate supervision signals. In *Proceedings of the 14th ACM international conference on web search and data mining*. 553–561.
- [4] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, and Ji-Rong Wen. 2022. Unikgqa: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph. *arXiv preprint arXiv:2212.00959* (2022).
- [5] Qimai Li, Zhichao Han, and Xiao-Ming Wu. 2018. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32.
- [6] Linhao Luo, Yuan-Fang Li, Reza Haffari, and Shirui Pan. 2024. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations*, Vol. 2024. 14400–14423.
- [7] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2025. Think-on-graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval augmented generation. In *International Conference on Learning Representations*, Vol. 2025. 52782–52806.
- [8] Costas Mavromatis and George Karypis. 2022. ReaRev: Adaptive reasoning for question answering over knowledge graphs. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. 2447–2458.
- [9] Costas Mavromatis and George Karypis. 2025. GNN-RAG: Graph neural retrieval for efficient large language model reasoning on knowledge graphs. In *Findings of the Association for Computational Linguistics: ACL 2025*. 16682–16699.
- [10] Kenta Oono and Taiji Suzuki. 2019. On asymptotic behaviors of graph cnns from dynamical systems perspective. *arXiv preprint arXiv:1905.10947* (2019).
- [11] Apoorv Saxena, Aditay Tripathi, and Partha Talukdar. 2020. Improving multi-hop question answering over knowledge graphs using knowledge base embeddings. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 4498–4507.
- [12] Jiaxin Shi, Shulin Cao, Lei Hou, Juanzi Li, and Hanwang Zhang. 2021. Transfnet: An effective and transparent framework for multi-hop question answering over relation graph. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 4149–4158.
- [13] Haitian Sun, Tania Bedrax-Weiss, and William Cohen. 2019. Pullnet: Open domain question answering with iterative retrieval on knowledge bases and text. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 2380–2390.
- [14] Haitian Sun, Bhuwan Dhingra, Manzil Zaheer, Kathryn Mazaitis, Ruslan Salakhutdinov, and William Cohen. 2018. Open domain question answering using early fusion of knowledge bases and text. In *Proceedings of the 2018 conference on empirical methods in natural language processing*. 4231–4242.
- [15] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *International Conference on Learning Representations*, Vol. 2024. 3868–3898.
- [16] Alon Talmor and Jonathan Berant. 2018. The web as a knowledge-base for answering complex questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. 641–651.
- [17] Keheng Wang, Feiyu Duan, Sirui Wang, Peiguang Li, Yunsen Xian, Chuantao Yin, Wenge Rong, and Zhang Xiong. 2023. Knowledge-driven cot: Exploring faithful reasoning in llms for knowledge-intensive question answering. *arXiv preprint arXiv:2308.13259* (2023).
- [18] Jason Wei, Xuezhì Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. 24824–24837 pages.
- [19] Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. 2016. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 201–206.
- [20] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do transformers really perform badly for graph representation? *Advances in neural information processing systems* 34 (2021), 28877–28888.
- [21] Jing Zhang, Xiaokang Zhang, Jifan Yu, Jian Tang, Jie Tang, Cuiping Li, and Hong Chen. 2022. Subgraph retrieval enhanced model for multi-hop knowledge base question answering. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: Long papers)*. 5773–5784.